ShannonRes

AI-Based SEM Image Restoration — Full Project Report

KLA AI Hackathon 2026 — Consolidated Technical Report (Architecture, Experiments, Diagnostics, and Open Items)

1. Project Overview

ShannonRes restores noisy, low-resolution SEM (Scanning Electron Microscopy) images of nanoscale surface structures back to clean, high-resolution images, for the KLA AI Hackathon's semiconductor metrology and inspection challenge. The task combines two degradations at once: denoising (removing speckle and Gaussian noise) and 2x super-resolution (128x128 input to 256x256 ground truth).

This report consolidates the full project journey: the original CNN baseline, the architectural diagnostic that motivated moving to NAFNet, three iterations of NAFNet tuning, a MambaIR feasibility investigation, latency optimization work, and a critical data-pipeline bug discovered late in the project that puts prior quality metrics under active re-verification. This is a working project report, not a final results document — open items are flagged explicitly throughout.

2. Architecture: From RestorationCNN to NAFNet

The project began with RestorationCNN, a 16-block residual CNN operating at a single spatial resolution throughout. Diagnostics (Section 4) showed this architecture had hit a capacity ceiling, losing fine texture in flat, detail-heavy regions. This motivated a move to NAFNet, a U-Net style encoder-bottleneck-decoder architecture that captures broader spatial context through downsampling while recovering fine detail via skip connections between matching encoder/decoder stages.

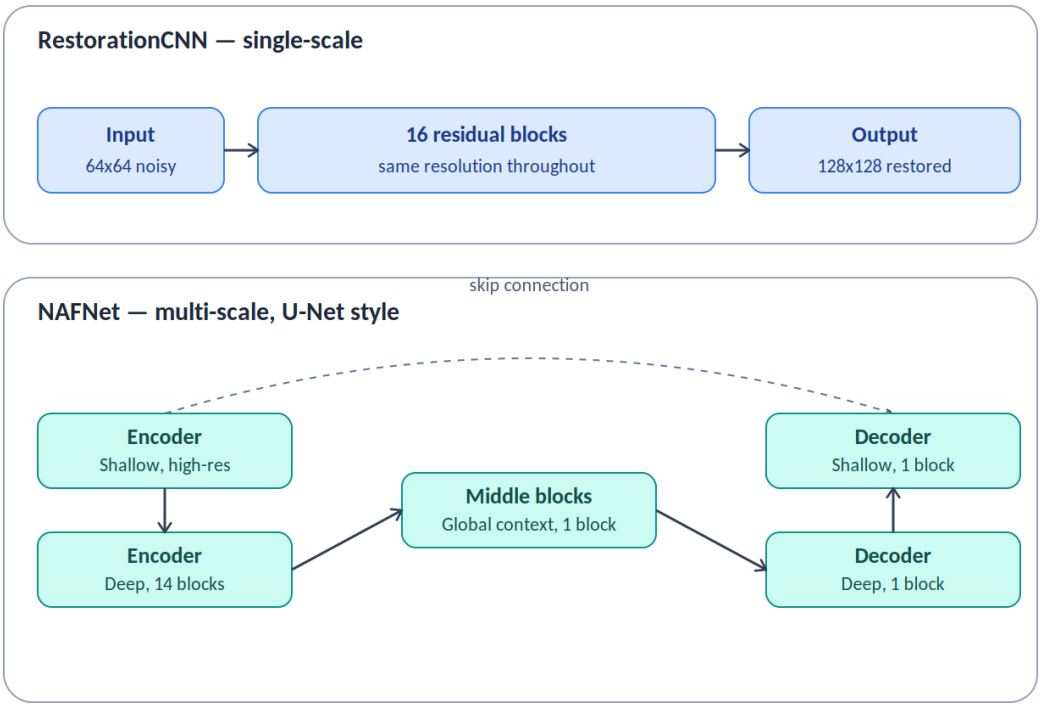


Figure 1. RestorationCNN (single-scale residual stack) versus NAFNet (multi-scale U-Net with skip connections).

NAFNet's name — Nonlinear Activation Free Network — refers to its internal block design, which replaces standard activation functions with a simpler gating mechanism (splitting channels and multiplying two halves together), aiming for strong quality at lower compute cost than a comparable transformer-based design.

3. Phase 1 & Phase 2: RestorationCNN Training

Phase 1 trained RestorationCNN on synthetic Gaussian noise, establishing an initial baseline. Phase 2 fine-tuned the model on a combined dataset of real-world noisy SEM images plus a subset of Phase 1 synthetic data, aiming to adapt to real noise characteristics while retaining general capability.

Phase 2 training ran to epoch 54 before early stopping (15-epoch patience, no validation PSNR improvement). The resulting gain over Phase 1 was marginal at the time it was measured (PSNR +0.14 dB, SSIM -0.004) — a difference within typical run-to-run noise, which motivated a deeper diagnostic rather than accepting the fine-tune at face value.

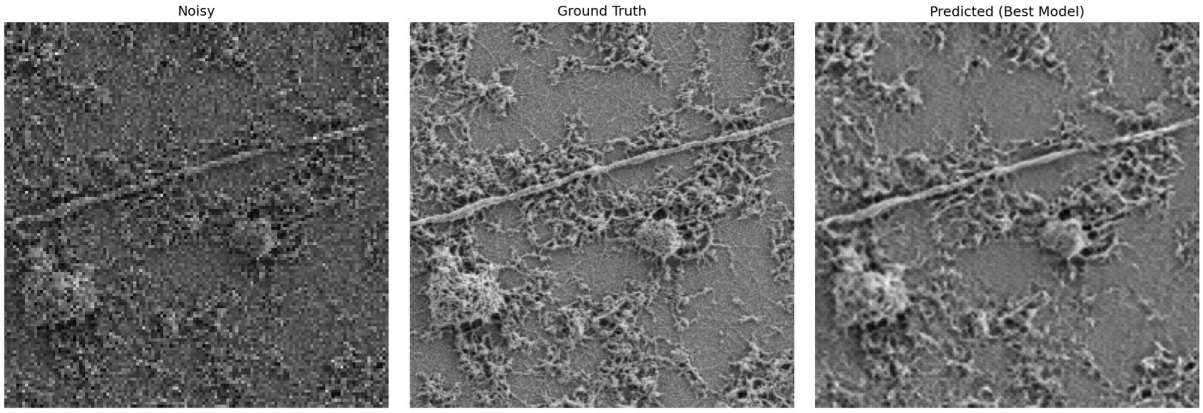


Figure 2. Phase 2 CNN: noisy input, ground truth, and prediction. Noise is largely removed, but fine filament structures are visibly smoothed relative to ground truth.

4. Diagnostic Investigation — Why the CNN Plateaued

A structured diagnostic was run to determine whether the Phase 2 plateau reflected overfitting, a data noise floor, or an architectural capacity ceiling.

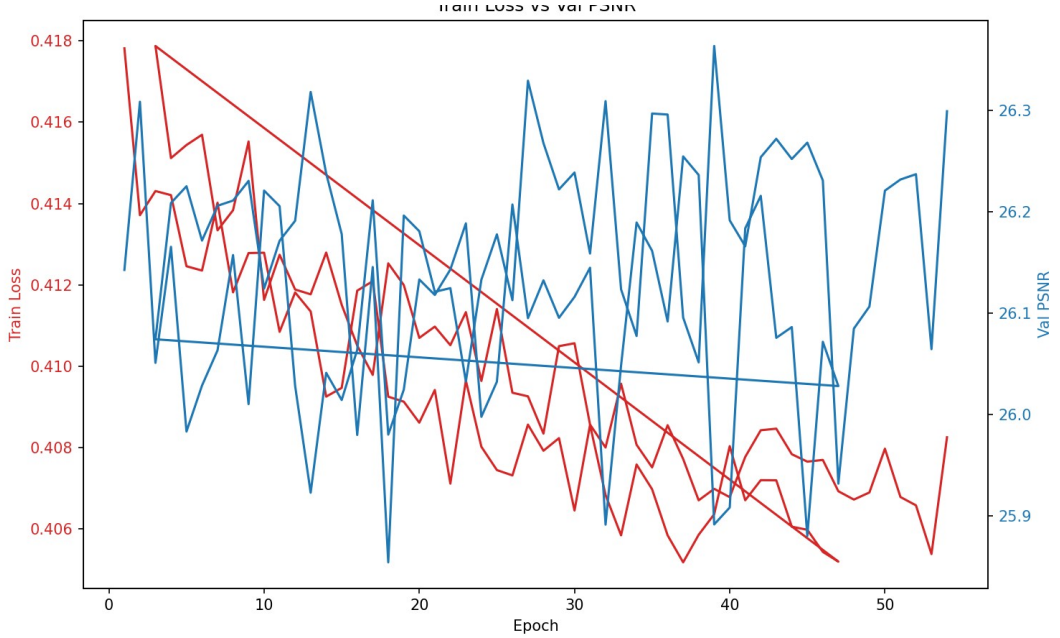


Figure 3. Training loss (red) vs. validation PSNR (blue) across 54 epochs. (Note: this curve's noise level is now understood to be partly explained by a later-discovered validation bug — see Section 8.)

Per-image error maps and a frequency-domain (high-frequency Laplacian) analysis across 10 validation samples found errors concentrated in fine detail and edges rather than spread as uniform noise — consistent with detail loss (blurring) rather than a random noise floor. Two representative samples are shown below.

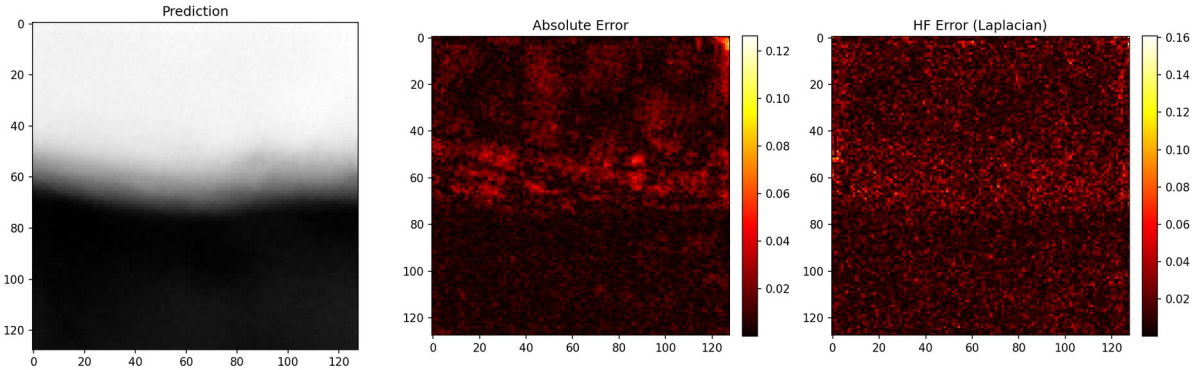


Figure 4. Validation sample showing blurred texture reconstruction relative to ground truth.

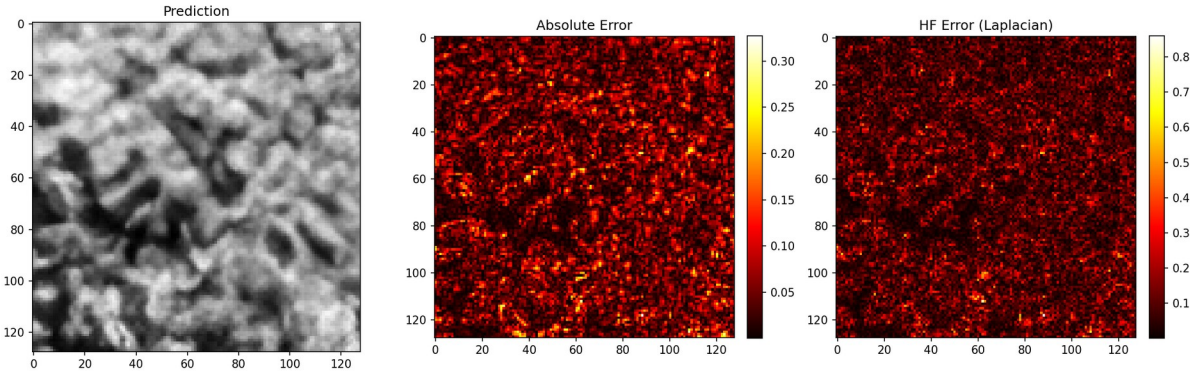


Figure 5. Validation sample in a flat, texture-heavy region — a consistent CNN weak point.

Conclusion at the time: architecture/capacity ceiling, not overfitting — motivating the move to NAFNet's multi-scale design.

5. NAFNet v1 — First Architecture Swap

NAFNet v1 used a lightweight, latency-prioritized configuration: width 32, with a strongly asymmetrical encoder/decoder (enc_blk_nums [1,1,1,14], dec_blk_nums [1,1,1,1]). Training resumed from epoch 38, found a new best at epoch 44, and early-stopped at epoch 59.

The new best validation PSNR for your NAFNet model (achieved at epoch 44) is **26.248**!
For comparison, before we resumed training, your previous best was 26.214 from epoch 38.
Here are all the validation metrics for that new best checkpoint:

- **PSNR**: `26.248`
- **SSIM**: `0.7348`
- **LPIPS**: `0.3086`

It's a small but nice improvement! The model weights in `weights/model_nafnet.pth` are now fully updated to this best-performing epoch.

Figure 6. NAFNet v1 training summary — best validation PSNR 26.248 at epoch 44.

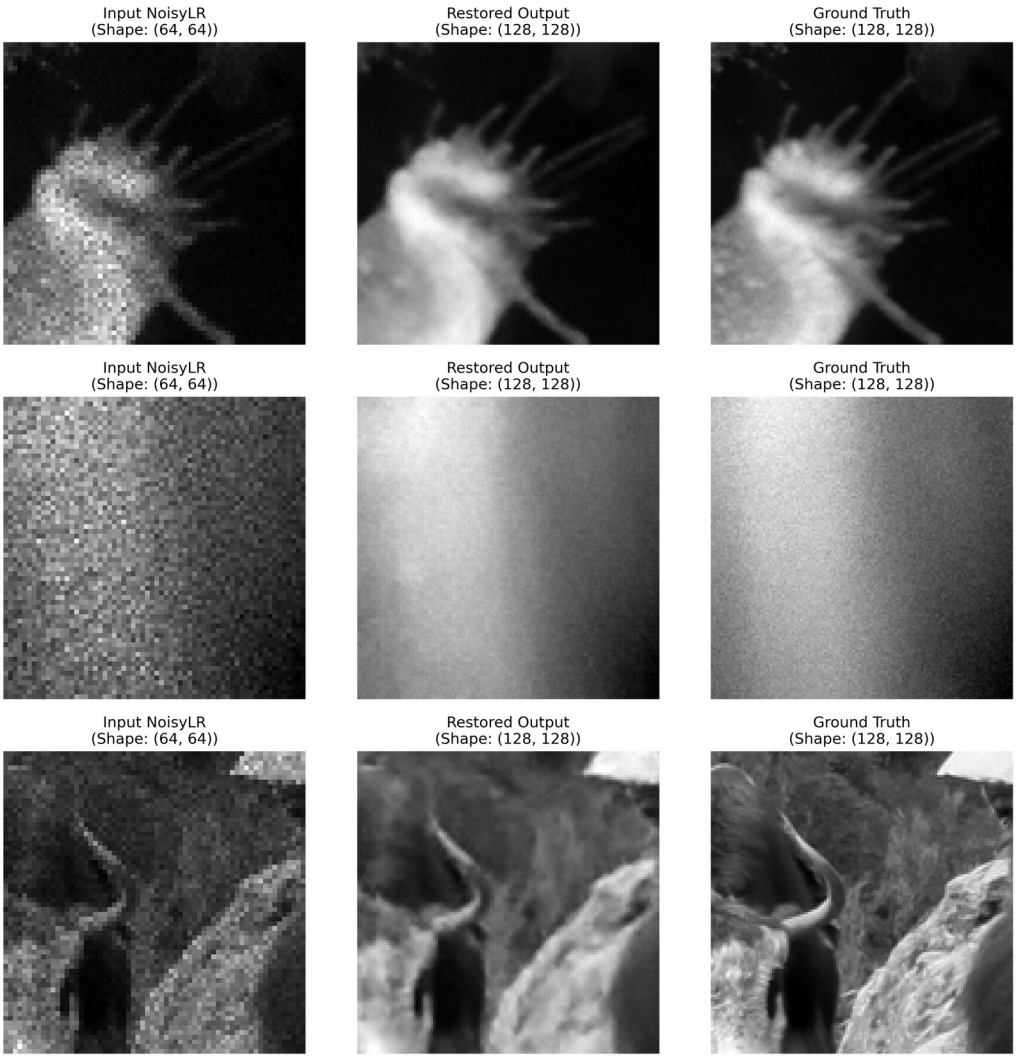


Figure 7. NAFNet v1 restoration samples. Structured content (spikes, ridgelines) tracked ground truth well; flat, texture-heavy regions (middle row) were noticeably oversmoothed.

Analysis at the time attributed the flat-region weakness to the decoder's near-total lack of capacity (only 1 block per stage versus 14 in the deepest encoder stage) — the stage of the network responsible for reconstructing fine detail during upsampling.

6. NAFNet v2 & v3 — Rebalancing and Pruning

NAFNet v2 rebalanced the decoder to mirror the encoder (dec_blk_nums [1,1,1,14]), which modestly improved PSNR/SSIM/LPIPS over v1 and the Phase 2 CNN — but latency ballooned to 40.09 ms eager / 25.70 ms compiled, roughly 10-16x over the project's self-imposed 2.47 ms reference budget (a budget based on the team's own RTX 4070 dev GPU, not the hackathon's official H100 benchmarking hardware).

Profiling identified LayerNorm and depthwise convolutions — both of which scale directly with block count — as the dominant cost. ONNX Runtime export (CUDAExecutionProvider) cut NAFNet v2's latency substantially, to 12.94 ms — proving PyTorch eager/Inductor mode was leaving significant performance on the table on this Windows environment, though still short of the reference budget.

NAFNet v3 pruned the decoder/encoder from [1,1,1,14] to [1,1,1,4] to directly attack the profiled bottleneck. This dropped ONNX latency further, to 5.39 ms (185 FPS) — very close to real-time — but PSNR fell to 26.237 and SSIM to 0.732, below both NAFNet v2 and the Phase 2 CNN, illustrating a direct, quantified quality/speed tradeoff along this architecture family.

7. MambaIR Investigation

MambaIR (a state-space/Mamba-based restoration architecture) was investigated as a third architecture family, motivated by its linear-complexity global receptive field — relevant given NAFNet's LayerNorm/conv-driven latency cost. The official implementation depends on mamba_ssm and causal_conv1d, which require compiling native CUDA kernels.

This build failed in the project's Windows environment (missing nvcc, no compatible prebuilt wheel for the exact PyTorch/CUDA/Python combination in use). A depthwise-convolution fallback was trained and benchmarked instead (PSNR 26.113, latency 2.78 ms) — fast, but this result does not reflect the real Mamba selective-scan mechanism and should not be read as a verdict on MambaIR's actual potential for this task. Testing the real architecture requires a Linux or WSL2 environment with nvcc available.

8. Critical Finding: Validation Pipeline Bug

Late in the project, a targeted audit was run to confirm the dataset pipeline matched the hackathon's actual resolution pairs (128->256 and 256->512). This surfaced a significant, previously unknown issue affecting every quality metric reported up to that point:

- The dataset's __getitem__ method applied a random crop to gt_crop_size (128) unconditionally — including during validation — meaning every PSNR/SSIM/LPIPS score reported in every prior comparison was computed on a different random 64x64 (input) / 128x128 (target) patch, not the full 128x128 -> 256x256 image pair.
- Inference latency numbers are unaffected (benchmark.py used a fixed-size dummy tensor, independent of this bug) and remain valid.
- A search of all provided dataset folders confirmed zero 256->512 pairs exist in the project; only 128->256 pairs are provided for training/testing. 256->512 may be exclusive to the organizers' hidden final test set.

The dataset loader has since been fixed to bypass random cropping during validation (via an is_val flag), and all six trained checkpoints are being re-evaluated on full, fixed images as of this report. An initial re-evaluation pass produced a counterintuitive result — the original Phase 1 CNN outperforming every later model — which

is itself under active verification for a second possible bug (inconsistent validation sets across checkpoints) before being accepted as a real finding.

9. Consolidated Results (Status: Under Re-Verification)

The table below consolidates every checkpoint trained so far. Quality metrics (PSNR/SSIM/LPIPS) predate the validation bug fix in Section 8 and are not currently trustworthy as an architecture comparison — they are included for project history, not as a final verdict. Latency figures are independently confirmed valid.

Model	PSNR	SSIM	LPIPS	Best Latency	Status
Phase 1 CNN	26.06–26.16	0.740–0.770	0.300	2.47 ms	Under re-verification
Phase 2 CNN	26.30	0.766	0.284	2.47 ms	Under re-verification
NAFNet v1	26.248	0.735	0.309	24.62 ms	Under re-verification
NAFNet v2	26.3145	0.739	0.297	12.94 ms (ONNX)	Under re-verification
NAFNet v3 (pruned)	26.237	0.732	—	5.39 ms (ONNX)	Under re-verification
MambaIR (fallback)	26.113	0.715	0.339	2.78 ms	Not real Mamba core

A corrected, unified re-evaluation (single fixed validation set, full images, applied identically to all six checkpoints) is in progress and will supersede this table.

10. Open Items & Next Steps

- Complete the corrected, unified re-evaluation of all six checkpoints on full 128x128->256x256 images with one shared validation split, and confirm or overturn the current provisional ranking.
- Resolve the 256->512 data question definitively — confirm with organizers whether this pairing is training-set-absent by design or requires a separate download.
- Re-benchmark latency expectations against H100-class hardware rather than the team's RTX 4070 dev GPU, since the hackathon's official scoring uses H100 and treats speed as a tiebreaker rather than a hard cutoff.
- Test real MambaIR (not the depthwise-conv fallback) in a Linux/WSL2 environment with nvcc available.
- Validate that no restoration model introduces systematic measurement bias (e.g. feature shrinkage) before any deployment-oriented claims are made — an industry-standard concern for SEM-based metrology specifically.
- Test out-of-distribution generalization, since the hackathon's held-out test set explicitly includes samples from sources not seen in training.
- Build the mandatory standalone CLI evaluation script and package the training script/notebook and environment spec ahead of the submission deadline.

11. Honest Summary

This project has produced a working, iteratively-diagnosed restoration pipeline with three architecture families explored, a documented and quantified latency/quality tradeoff across NAFNet configurations, and — importantly — the discovery and correction of a real data-pipeline bug that had been silently invalidating quality comparisons for several iterations. No model has yet been confirmed as the best-performing option under correct evaluation conditions; that determination is the immediate next milestone, not a settled conclusion.